

SDEV 2351

DAY 9: ADVANCED ROUTING PATTERNS

FALLBACK ROUTES, CATCH-ALL PARAMETERS + LAB 1 ASSIGNMENT

Session Objectives

1. Implement fallback routes to handle unmatched URLs with a custom 404 page
2. Use catch-all route parameters to capture flexible URL paths
3. Describe route precedence rules when multiple templates could match
4. Understand Lab 1 requirements, rubric, and starter code structure

Today's Schedule

PHASE	TIME	WHAT'S HAPPENING
Connecting	5 min	Routing recap + today's structure
Exploring	20 min	Fallback routes + catch-all + route precedence
Practicing	40 min	Custom 404 + catch-all + routing quiz
Assessing	10 min	Module 1 check + Micro-Assessment #1 reminder (due today)
Lab 1 Assignment	30 min	Spec walkthrough + starter code orientation

PHASE 1 · 5 MINUTES

CONNECTING

Completing the routing picture

Routing So Far

We've covered: `@page` directives, route parameters, constraints, `NavigationManager`, `NavLink`, route guards, state persistence.

One gap remains: What happens when someone types a URL that doesn't match any page?

Today we add **fallback routes** and **catch-all parameters** to complete the routing picture. Then Lab 1 drops in the last 30 minutes.

PHASE 2 · 20 MINUTES

EXPLORING

Fallback routes and catch-all parameters

The Trap: **<NotFound>** Is Dead Code

Every Blazor tutorial older than .NET 8 shows you this:

```
<Router AppAssembly="typeof(Program).Assembly">
  <Found Context="routeData">
    <RouteView RouteData="routeData" />
  </Found>
  <NotFound>          @* compiles fine – never runs *@
    <LayoutView Layout="typeof(Layout.MainLayout)">
      <h3>Page Not Found</h3>
    </LayoutView>
  </NotFound>
</Router>
```

In a **Blazor Web App**, the framework ignores it. It is kept only so old code still compiles — it fires on neither the server render nor the browser one.

It compiles. It never fires. Nothing warns you.

Fallback Routes — the real mechanism

Three pieces. All required.

```
// Program.cs — re-runs the pipeline against /not-found,  
// KEEPING the 404 status. (ReExecute, not "ReExecution")  
app.UseStatusCodePagesWithReExecute("/not-found");
```

```
@* Components/Pages/NotFound.razor — an ORDINARY routed page *@  
@page "/not-found"  
  
<div class="alert alert-warning mt-4">  
    <h3>Page Not Found</h3>  
    <a href="/" class="btn btn-primary">Go Home</a>  
</div>
```

```
@* Components/Routes.razor — what the router shows when it has nothing *@  
<Router AppAssembly="typeof(Program).Assembly"  
    NotFoundPage="typeof(Pages.NotFound)">
```

It's a normal page, so it gets `MainLayout` from `DefaultLayout` — **no** `<LayoutView>` **needed**.

Why Three Pieces?

Your app renders twice.

1. **Server render.** Routing 404s → the middleware re-runs the pipeline against `/not-found` → your styled page, with a real 404 status.

2. **Browser render.** Blazor connects, and the router runs **again** — against the URL you actually typed. `/nonexistent` still matches nothing.

`NotFoundPage` is what the router renders on that second pass.

Leave it out and Blazor substitutes its own bare `Not found` — which lands on top of your styled page a moment after it appears.

The symptom: your 404 page **flashes, then vanishes.**

The Two Kinds of 404

	/NONEXISTENT	/PRODUCTS/999
Route matched?	No	Yes — {id:int}
Who detects it?	ASP.NET Core routing	Your page
What handles it	UseStatusCodePagesWithReExecute + NotFoundPage	NavigationManager.NotFound() + NotFoundPage

999 is an integer — routing matched the page and returned 200. The URL is fine; the **product** is missing. Only your page knows.

The Two Kinds of 404 — One Shared Page

```
if (!Catalog.TryGetValue(Id, out _name))  
{  
    Navigation.NotFound();    // renders NotFoundPage, returns 404  
}
```

NotFoundPage is in both columns. It is not the missing-data tool — it is the page the router falls back to, whichever way it got there.

Two kinds of 404. Two detectors. One shared page.

Catch-All Parameters

A catch-all page with `@page "/docs/{*slug}"`

The `{*paramName}` syntax captures everything after the slash, including nested segments:

- `@page "/docs/{*slug}"` matches:
 - `/docs/getting-started` -- slug = `"getting-started"`
 - `/docs/api/v2/endpoints` -- slug = `"api/v2/endpoints"`

Use catch-all for CMS-style content, documentation systems, or any URL pattern where you don't know the structure in advance.

Route Precedence

When multiple templates could match, Blazor picks the most specific:

PRIORITY	ROUTE TYPE	EXAMPLE	MATCHES
1 (highest)	Literal	<code>/products</code>	Only <code>/products</code>
2	Constrained parameter	<code>/products/{id:int}</code>	<code>/products/42</code>
3	Unconstrained parameter	<code>/products/{name}</code>	<code>/products/laptop</code>
4 (lowest)	Catch - all	<code>/[*path]</code>	Everything else

More specific routes always win. A constrained parameter beats an unconstrained one; both beat a catch-all.

Optional Parameters

`{id:int?}` makes a parameter optional. The component must handle the null case.

```
@page "/settings/{id:int?}"

@code {
    [Parameter] public int? Id { get; set; }

    protected override void OnInitialized()
    {
        if (Id is null)
            // Show settings list
        else
            // Show specific setting
    }
}
```

Full code: [see reference-sheet.md](#) or your IDE

PHASE 3 · 40 MINUTES

PRACTICING

Fallback routes, catch-all, and routing consolidation

Practice 1: Fallback Route (10 min)

GUIDED

Starter provides: No fallback route — `/nonexistent` gives the browser's error page.

You implement — all three pieces are required:

- `Program.cs`: `app.UseStatusCodePagesWithReExecute("/not-found");`
- `Components/Pages/NotFound.razor`: `@page "/not-found"`, styled 404 message, "Go Home" link
- `Components/Routes.razor`: `NotFoundPage="typeof(Pages.NotFound)"` on the `<Router>`

Test: Navigate to various nonexistent URLs — and **wait a second** on the 404 page. The missing-`NotFoundPage` failure only appears once Blazor connects.

Practice 2: Catch-All Route (20 min)

INDEPENDENT

Starter provides: A `Colors.razor` page that needs a catch-all route. The lookup dictionary (`red / green / blue` → hex) is **already written** -- the routing is the exercise.

You implement:

- One `@page "/colors/{*slug}"` directive -- a catch-all also matches **zero** segments, so bare `/colors` lands here with `Slug = null`
- A `[Parameter] public string? Slug` (that null is why it must be `string?`)
- Three branches: list all colors / show name + hex + swatch / "Color Not Found" printing the slug

Then try `/colors/red/dark` -- the slug prints as `red/dark`. Both segments, one string. Without the star, that URL would not reach the page at all.

Practice 3: Routing Consolidation Quiz (10 min)

WRITTEN

No coding. Match each URL to the route template that handles it:

URL	ROUTE TEMPLATE
<code>/products</code>	<code>@page "/products"</code> (literal)
<code>/products/42</code>	<code>@page "/products/{id:int}"</code> (constrained)
<code>/products/laptop</code>	<code>@page "/products/{name}"</code> (unconstrained)
<code>/docs/api/v2/auth</code>	<code>@page "/docs/{*slug}"</code> (catch-all)
<code>/settings</code> (no id)	<code>@page "/settings/{id:int?}"</code> (optional)
<code>/nonexistent</code>	Fallback route (<code>NotFound.razor</code>)

PHASE 4 · 10 MINUTES

ASSESSING

Module 1 check + Micro-Assessment #1 reminder

Module 1 Check + MA #1 due today (10 min)

Module 1 comprehension + confidence check (8 min)

- Quick cold call: why `<NotFound>` does nothing, the two kinds of 404, pure reducers, when route guards fire
- Exit ticket: the one Module 1 topic you feel least ready for + one you feel solid on (steers Day-10 review)

Micro-Assessment #1 reminder (2 min)

2% — CO6 — TAKE-HOME

- **MA #1 — Git Branching Workflow** was assigned Day 5 and is **due today**.
- Do it in your **GitHub Classroom repo**: branch → commit → push → PR → resolve a conflict → merge.
- Commit your **proof-of-self file** to that repo, then **paste your repo URL** into the MA #1 Assignment folder in Brightspace so your instructor can find it.
- Pre-submit checklist: the **Section 14 self-check** in [Git Branching — Your Complete Lab Guide](#).

Summary

Today you learned to:

1. **Implement fallback routes** to handle unmatched URLs with a custom 404 page
2. **Use catch-all route parameters** to capture flexible URL paths
3. **Describe route precedence rules** -- literal > constrained > unconstrained > catch-all

30 MINUTES

LAB 1 ASSIGNMENT

Navigation + State Management

Lab 1 Overview

What you'll build: A navigation + state management application using Fluxor

Course Outcomes: CO1 (Routing), CO2 (State Management), CO6 (Team Collaboration)

Technical requirements:

- Fluxor for state management
- Route guards
- At least one persistence pattern
- BYSResult in service layer

Rubric Breakdown — Four-Bucket Gateway-Class

BUCKET	WEIGHT	WHAT IT MEASURES
Architecture (gateway)	30%	Pattern fidelity. Sub-check failures cap the bucket.
Functionality	40%	Behavioral correctness + hidden + visible auto-test pass rate.
Individual Contribution	20%	Walkthrough recording (8%) + student-authored tests (4%) + per-student CO6 git signals (8%).
Peer Evaluation	10%	Confidential teammate ratings, cross-checked against git evidence.

Teams: 5-6 students, formed on Day 10

Due: Day 18

Starter Code Structure

The starter repository includes:

- Project structure with base Fluxor setup
- README with feature descriptions and acceptance criteria
- Each requirement has clear acceptance criteria

You do not clone today. Cloning happens on Day 10 during the guided Git setup with your team.

What's Next

Day 10: Module 1 Review + Team Formation + Lab Prep

- No new instruction
- Review all Module 1 concepts
- Form teams and assign roles
- Plan your Lab 1 architecture and claim features

Come prepared with questions about the lab spec.

Quick self-check: which Module 1 topic do you feel least prepared for? Bring that question to Day 10.